

Subset Sum Problem with Sorted Input Pruning

Introduction

The Subset Sum Problem is a well-known combinatorial problem in computer science. The objective is to determine whether a subset of a given set of positive integers adds up exactly to a specified target value. Since the number of possible subsets grows exponentially with the size of the input, efficient search techniques are required. Backtracking with pruning helps reduce unnecessary exploration and improves performance.

Problem Description

Given a sorted set of positive integers and a target sum, determine whether there exists a subset whose elements add up exactly to the target value.

Inputs

- Sorted set of positive integers
- Target sum

Output

- True/False indicating whether a valid subset exists
- The valid subset if a solution is found

Solution Approach

The problem is solved using backtracking. At each step, an element may either be included in or excluded from the current subset. The algorithm maintains the current sum and the remaining sum of unprocessed elements. Pruning is applied whenever the current sum exceeds the target or when the remaining elements are insufficient to reach the target.

Analysis

The inclusion-exclusion strategy ensures that all feasible subsets are explored. The pruning conditions significantly reduce the search space by eliminating branches that cannot lead to a valid solution.

Correctness

The algorithm systematically explores every possible subset while avoiding impossible branches. Therefore, it correctly identifies whether a subset with the required sum exists.

Complexity

- Time Complexity: $O(2^n)$
- Space Complexity: $O(n)$

Conclusion

The Subset Sum Problem can be effectively solved using backtracking combined with pruning techniques. Pruning improves efficiency while maintaining correctness.